5_23_marzo

Refactoring
Come abbiamo detto fare refactoring vuol dire cambiare il codice per sistemare diversi aspetti: La complessità, l'eliminazione di bug, l'organizzazione e la leggibilità del codice, infatti scrivere codice leggibile e semplice è la chiave per scrivere un buon sistema software, infatti così facendo in futuro il software potrà essere ampliato o manutenzionato più semplicemente.
Il refactoring quindi non mira a cambiare i risultati che il software ci restituisce ma si limita a sistemare un codice già funzionante.

Come si fa refactoring?
Per farlo si usano diverse tecniche che possiamo vedere un po' come una serie di step da applicare in base alla parte di codice in cui vogliamo fare refactoring. Dopo aver finito si rieseguono tutti i test in modo che se si è compromesso qualcosa si è subito avvertiti e si può sistemare.

Quando fare refactoring
Non sempre è necessario farlo, però si può capire se fare refactoring è utile quando si presentano diversi sintomi nel codice (bad smell o code smell):

- Quando è presente molto codice duplicato:
  è infatti inutile duplicare il codice molte volte, sarebbe meglio creare un metodo che fa solo poche cose e che può essere richiamato all'occorrenza
- Quando si hanno metodi e classi di dimensioni troppo grandi e con troppe linee di codice
- Quando si ha una lista di parametri per un metodo che è troppo lunga
- Per cambiamenti divergenti:
  In pratica nella buona programmazione ad ogni classe spetterebbe un compito solo, altrimenti rischia di diventare come un coltellino svizzero (versatile ma difficile per fare manutenzione), i cambiamenti divergenti sono appunto quando occorre modificare uno dei tanti comportamenti della classe rischiando di generare nuovi bug (oltre ad essere un lavoro più difficile)

Vediamo adesso le varie tecniche di refactoring:

Tecniche di refactoring

ESTRAI METODO
Obbiettivo:

- L'obbiettivo è prendere un gruppo di linee di codice di un metodo e farlo diventare un altro metodo a se stante. Avere metodi piccoli favorisce il riuso di questi ultimi e

rende il codice più semplice da capire

Procedimento:

- Creare un nuovo metodo il cui nome comunica l'intenzione del metodo
- Copiare il codice estratto dal metodo sorgente al nuovo metodo
- Guardare se il codice estratto ha variabili locali al metodo sorgente e far diventare tali variabili parametri del metodo nuovo
- Se alcune variabili sono usate solo all'interno del codice estratto farle diventare variabili temporanee del nuovo metodo
- Se una variabile locale al metodo sorgente è modificata dal codice estratto, vedere se è possibile far sì che il metodo estratto sia una query e assegnare il risultato della chiamata alla variabile locale del metodo sorgente
- Sostituire nel metodo sorgente il codice estratto con una chiamata al metodo nuovo

Esempio:

```java
public class Ordine {
    private double importo;
    public double getImporto() {
        return importo;
    }
}
// prima del refactoring
public class Stampe {
    private ArrayList<Ordine> ordini;
    private String nome = "Mike";

    public void printDovuto() {
        Iterator<Ordine> i = ordini.iterator();
        double tot = 0;

        // scrivi banner
        System.out.println("----------------");
        System.out.println("- Cliente Dare -");
        System.out.println("----------------");

        // calcola totale
        while (i.hasNext()) {
            Ordine o = i.next();
            tot += o.getImporto();
        }

        // scrivi dettagli
        System.out.println("nome: " + nome);
        System.out.println("tot: " + tot);
    }
}
```

```java
public class Stampe2 { // dopo il refactoring
    private ArrayList<Ordine> ordini;
    private String nome = "Mike";

    public void printDovuto() {
        printBanner();
        double tot = getTotale();
        printDettagli(tot);
    }

    public double getTotale() {
        Iterator<Ordine> i = ordini.iterator();
        double tot = 0;
        while (i.hasNext()) {
            Ordine o = i.next();
            tot += o.getImporto();
        }
        return tot;
    }

    public void printBanner() {
        System.out.println("----------------");
        System.out.println("- Cliente Dare -");
        System.out.println("----------------");
    }

    public void printDettagli(double somma) {
        System.out.println("nome: " + nome);
        System.out.println("tot: " + somma);
    }
}
```

8 Prof. Tramontana - Marzo 2025

Sulla sinistra abbiamo un metodo (printDovuto) lungo e complesso a cui può essere fatto refactoring, in particolare ho individuato 3 compiti differenti che questo metodo ha (scrivi banner, calcola totale e scrivi dettagli), compiti che posso dividere in 3 metodi più piccoli e semplici e poi usare printDovuto per richiamarli

TEMP CON QUERY

Questo metodo conviene usarlo solo se la variabile temporanea è usata una volta sola all'interno del metodo.
Obbiettivo:

- Posso prendere un' espressione che mi ritorna un valore e farla diventare un metodo che restituisce quel valore (usata quando si usano variabili temporanee (temp)), questo perché è probabile che poi questo metodo possa essere usato in un altro punto del programma

Procedimento:

- Cercare una variabile temporanea assegnata solo una volta
- Dichiarare temp final
- Compilare (per verificare che è assegnata una volta sola)
- Estrarre la parte destra dell'assegnazione e creare un metodo
  Esempio:

```
private double quantita, prezzo;

public double getPrezzo1() {
   double prezzoBase = quantita * prezzo;
   double sconto;
   if (prezzoBase > 1000) sconto = 0.95;
   else sconto = 0.98;
   return prezzoBase * sconto;
}
```

- Diventa, sostituendo entrambe le variabili `prezzoBase` e `sconto`

```
private double quantita, prezzo;

public double getPrezzo2() {
   return prezzoBase() * sconto();
}

private double prezzoBase() {
   return quantita * prezzo;
}

private double sconto() {
   if (prezzoBase() > 1000) return 0.95;
   return 0.98;
}
```

12 Prof. Tramontana - Marzo 2025

DIVIDI VARIABILE TEMP

Obbiettivo:

- Se ho una variabile temp assegnata più volte (non in un loop o usata per accumulare valori) allora posso usare più variabili per separare meglio e rendere più leggibile il codice (è anche buona programmazione così come per classi e metodi che le variabili abbiano un solo compito)

Procedimento:

- Cambiare il nome della variabile temp al momento della dichiarazione e alla sua prima assegnazione
- Dichiarare la nuova temp come final
- Cambiare tutti i riferimenti a temp fino alla sua seconda assegnazione

- Dichiarare una nuova temp per la seconda assegnazione
- Compilare e testare
- Ripetere per singoli passi, ogni passo rinomina una dichiarazione e cambia i riferimenti fino alla prossima assegnazione
  Esempio:
  - Usare una variabile separata per ciascun assegnamento

    ```
    double temp = 2 * (height + width);
    System.out.println(temp);
    temp = height * width;
    System.out.println(temp);
    ```

  - Diventa

    ```
    final double perim = 2 * (height + width);
    System.out.println(perim);
    final double area = height * width;
    System.out.println(area);
    ```

SPOSTA METODO

Obbiettivo:

- Occorre usare questa tecnica quando dentro una classe ho un metodo che usa moltissimi attributi di un'altra classe in proporzione a quelli che usa appartenenti alla sua classe. Spostare un metodo rende le responsabilità delle classi più chiare

Procedimento:

- Esaminare attributi e metodi usati dal metodo da spostare per decidere se spostare anche questi (spostarli se sono usati solo dal metodo trasferito)
- Controllare che il metodo da spostare non sia dichiarato anche in superclassi e sottoclassi
- Dichiarare il metodo nella classe target
- Copiare il metodo nella classe target e sistemare le chiamate a metodi della classe origine e della classe target
- Richiamare il metodo nella classe target con l'opportuno riferimento
- Decidere se rimuovere il metodo originario o tenerlo per delegare (chiamare al suo interno il nuovo metodo nell'altra classe)

ESTRAI CLASSE

Obbiettivo:

- Occorre usare questa tecnica quando una classe svolge dei compiti che dovrebbero svolgere 2 classi (1 compito per classe). Es. la classe Persona ha attributi per tenere prefisso e numero di telefono

Procedimento:

- Decidere quali responsabilità separare
- Creare una nuova classe per tali responsabilità
- Inserire una dipendenza tra la vecchia e la nuova classe: un attributo nella vecchia classe che tiene una istanza della nuova classe
- Spostare attributi e metodi dalla vecchia classe alla nuova classe, iniziando dai metodi di più basso livello